

Universidad San Francisco de Quito

Sytem Design

PSet #3

1. Contexto

Una fintech necesita una plataforma interna para gestionar incidentes operativos, tareas de seguimiento y notificaciones a usuarios internos. Hoy estos procesos se manejan con correo, hojas de cálculo y mensajería informal, lo que genera poca trazabilidad, mala asignación de responsables, baja visibilidad del estado real de los incidentes y notificaciones inconsistentes.

Deben construir **OpsCenter**, una plataforma interna que resuelva ese problema.

2. Objetivo del PSet

Diseñar e implementar una aplicación integrada que demuestre dominio de:

- modelado del dominio,
- arquitectura hexagonal,
- uso de ORM en la capa de repositorio,
- separación entre DTOs, dominio y persistencia,
- autenticación y autorización,
- patrones de diseño,
- UML,
- Docker,
- frontend con Streamlit,
- y trabajo colaborativo profesional en GitHub.

3. Qué deben construir

Construirán una aplicación con dos componentes principales:

Backend

Responsable de:

- autenticación,
- autorización,
- lógica de negocio,
- persistencia,
- eventos del sistema,
- notificaciones,
- APIs.

Frontend

Construido con **Streamlit**, debe permitir:

- login,
- visualización de incidentes,
- creación de incidentes,
- visualización y actualización de tareas,
- visualización de notificaciones.

4. Módulos obligatorios

El sistema debe incluir obligatoriamente:

- autenticación,
- usuarios con roles y permisos,
- incidentes,
- tareas,
- notificaciones,
- eventos del sistema,
- persistencia con ORM,
- frontend en Streamlit.

5. Roles del sistema

Deben existir al menos estos roles:

- **ADMIN**
- **SUPERVISOR**
- **OPERATOR**

Permisos esperados

ADMIN

Puede ver todo, gestionar usuarios o roles si deciden implementarlo, asignar incidentes, cambiar estados y consultar todas las notificaciones.

SUPERVISOR

Puede ver incidentes y tareas del sistema, asignar incidentes, cambiar estados y consultar notificaciones.

OPERATOR

Puede crear incidentes, ver incidentes creados por él o asignados a él, ver y actualizar tareas asignadas y consultar sus notificaciones.

6. Entidades (pueden extenderlas)

User

Debe tener al menos:

- `id`
- `name`
- `email`
- `password` o `hashed_password`
- `role`

Incident

Debe tener al menos:

- `id`

- `title`
- `description`
- `severity`
- `status`
- `created_by`
- `assigned_to`
- `created_at`

Task

Debe tener al menos:

- `id`
- `incident_id`
- `title`
- `description`
- `status`
- `assigned_to`
- `created_at`

Notification

Debe tener al menos:

- `id`
- `recipient`
- `channel`
- `message`
- `event_type`
- `status`
- `created_at`

7. Casos de uso

7.1 Login

El usuario debe poder autenticarse con email y password.
Si las credenciales son válidas, el sistema debe permitir acceso.
Si no lo son, debe rechazar la autenticación.

7.2 Crear incidente

Un usuario autorizado debe poder crear un incidente con:

- título,
- descripción,
- severidad.

Reglas:

- **title** no puede ser vacío
- **description** no puede ser vacía
- **severity** debe ser válida
- el incidente inicia en estado **OPEN**

7.3 Consultar incidentes

- **Operator**: solo ve incidentes creados por él o asignados a él
- **Supervisor**: ve todos los incidentes operativos
- **Admin**: ve todos los incidentes

7.4 Ver detalle de incidente

Debe mostrarse al menos:

- título,
- descripción,
- severidad,
- estado,
- creador,
- asignado,
- fecha de creación,
- tareas asociadas

7.5 Asignar incidente

Solo **SUPERVISOR** y **ADMIN** pueden asignar incidentes a usuarios existentes.

7.6 Cambiar estado de incidente

Deben modelar un flujo de estados.

Estados sugeridos:

- **OPEN**

- ASSIGNED
- IN_PROGRESS
- RESOLVED
- CLOSED

7.7 Crear tarea asociada a incidente

Debe ser posible crear tareas ligadas a un incidente existente.

7.8 Consultar tareas

- **Operator:** ve sus tareas asignadas
- **Supervisor:** ve tareas del sistema o del equipo
- **Admin:** ve todas las tareas

7.9 Cambiar estado de tarea

Estados sugeridos:

- OPEN
- IN_PROGRESS
- DONE

7.10 Generar notificaciones desde eventos

Eventos mínimos sugeridos:

- INCIDENT_CREATED
- INCIDENT_ASSIGNED
- INCIDENT_STATUS_CHANGED
- TASK_CREATED
- TASK_DONE

7.11 Consultar notificaciones

- **Operator:** ve sus notificaciones
- **Supervisor:** ve las suyas y/o las del equipo
- **Admin:** puede ver todas

8. Patrones y uso esperado

Observer

Debe usarse para el manejo de eventos del sistema.

Debe existir:

- un publisher o event bus,
- una abstracción de observer,
- al menos 2 observers concretos.

Command

Debe usarse para encapsular el envío de notificaciones.

Debe existir:

- una abstracción `Command`,
- al menos 2 comandos concretos,
- el método `execute()`.

State

Debe usarse para controlar el ciclo de vida de una entidad.

La recomendación es aplicarlo a `Incident`.

Template Method

Debe usarse para construir mensajes de notificación según el canal.

Debe existir:

- una clase abstracta base,
- un template method,
- al menos 2 implementaciones concretas.

Factory

Debe usarse para crear entidades o comandos de manera consistente y centralizar validaciones.

Patrón creacional adicional

Deben implementar uno adicional y justificarlo en el README.
La opción más natural para este proyecto es **Abstract Factory** en el módulo de notificaciones.

9. Arquitectura requerida

Se debe seguir una **arquitectura hexagonal** con alta cohesión y bajo acoplamiento.

Domain

Aquí deben vivir:

- entidades,
- enums,
- reglas de negocio,
- estados,
- comandos,
- templates,
- observers base,
- interfaces de repositorio.

Application

Aquí deben vivir:

- casos de uso,
- services o facades,
- DTOs,
- coordinación del flujo.

Infrastructure

Aquí deben vivir:

- implementación de repositorios,
- modelos ORM,
- conexión a base de datos,
- autenticación,
- providers de notificación,
- event bus concreto.

API

Aquí deben vivir:

- rutas,
- dependencias,
- guards,
- endpoints.

Frontend

Aquí deben vivir:

- la app de Streamlit,
 - vistas,
 - formularios,
 - consumo de API.
-

10. Requisitos de persistencia

El backend debe usar **ORM obligatoriamente** en la implementación de repositorios.

Reglas

- el ORM debe vivir en **Infrastructure**
- el dominio no puede depender del ORM
- los endpoints no pueden consultar ORM directamente
- no se pueden exponer modelos ORM directamente por la API

Separación esperada

- **DTOs:** contrato de entrada y salida
 - **Dominio:** entidades y reglas
 - **ORM:** persistencia concreta
-

11. Backend

El backend debe exponer endpoints suficientes para soportar el sistema.

Endpoints sugeridos:

- `POST /login`
- `GET /me`
- `GET /incidents`
- `POST /incidents`
- `GET /incidents/{id}`
- `PATCH /incidents/{id}/assign`
- `PATCH /incidents/{id}/status`
- `GET /tasks`
- `POST /tasks`
- `PATCH /tasks/{id}/status`
- `GET /notifications`

No es obligatorio usar exactamente estos paths, pero sí cubrir esta funcionalidad.

12. Frontend

El frontend en Streamlit debe incluir como mínimo:

- pantalla de login,
- vista de incidentes,
- formulario para crear incidente,
- vista de tareas,
- vista de notificaciones.

Puede adaptar la experiencia según el rol, pero la autorización debe implementarse en backend.

13. Docker

El proyecto debe correr con:

`docker compose up --build`

Deben existir al menos estos servicios:

- `db`
- `api`

- `ui`

14. UML obligatorio

Deben entregar en `/docs`:

Use Case Diagram

Con los actores:

- Admin
- Supervisor
- Operator

Y los principales casos de uso del sistema.

Class Diagram

Debe incluir al menos:

- User
- Role
- Incident
- Task
- Notification
- State
- Observer/EventBus
- Command
- Template
- Factory
- interfaces o repositorios relevantes

Sequence Diagram

Debe modelar el siguiente flujo:

Operator crea incidente → backend persiste → se publica evento → observer genera notificación → template construye mensaje → command ejecuta envío → notificación actualiza estado

15. GitHub workflow

Todo el proyecto debe administrarse con un **GitHub Project**.

Board mínimo

Debe incluir al menos estas columnas:

- Backlog
- Todo
- In Progress
- In Review
- Done

Historias de usuario

El trabajo debe organizarse en historias de usuario.

Ejemplos:

- Como Operator, quiero crear un incidente para reportar una falla.
- Como Supervisor, quiero asignar incidentes para distribuir trabajo.
- Como usuario, quiero recibir una notificación cuando me asignen un incidente.

Issues

Cada integrante debe tener al menos **3 issues** propios.

Cada issue debe incluir:

- título,
- descripción,
- criterios de aceptación,
- label,
- assignee.

Pull Requests

Cada integrante debe abrir al menos **2 PRs**.

Cada PR debe:

- referenciar un issue,
- explicar lo implementado,
- recibir al menos 1 review.

Reviews

No se aceptan reviews vacíos como:

- ok
 - done
 - looks good
-

16. Entregables

Deben entregar:

- repositorio completo en GitHub,
- README,
- diagramas UML en [/docs](#),
- evidencia del GitHub Project,
- Demo funcional.

README

Debe explicar:

- qué problema resuelve el sistema,
 - arquitectura,
 - cómo correrlo,
 - cómo usarlo,
 - dónde aplicaron cada patrón,
 - justificación del patrón creacional adicional.
-

17. Escenarios para la demo

Durante la demo deben poder mostrar como mínimo:

1. login exitoso
 2. creación de un incidente desde frontend
 3. persistencia correcta del incidente
 4. publicación de un evento
 5. reacción de un observer
 6. construcción del mensaje con Template Method
 7. ejecución del envío con Command
 8. cambio de estado con State
 9. uso de Factory en la creación de una entidad o comando
 10. consulta de incidentes, tareas y notificaciones según rol
-

18. Restricciones

No se acepta:

- lógica de negocio en endpoints,
 - solución en un solo archivo,
 - patrones “solo por nombre”,
 - ausencia de autenticación,
 - ausencia de roles y permisos,
 - ausencia de frontend funcional,
 - dominio mezclado con ORM,
 - GitHub Project sin uso real,
 - issues sin mapear a historias de usuario,
 - PRs sin reviews.
-

19. Rúbrica

- Arquitectura y separación de capas — **10 pts**
- Uso correcto del ORM en repositorios — **10 pts**
- Patrones de comportamiento — **20 pts**
- Patrones creacionales — **10 pts**
- Backend — **10 pts**
- Frontend — **10 pts**
- UML — **5 pts**
- GitHub workflow y documentacion — **5 pts**
- Demo — **20 pts**